The Myth of the "Living" Document: Are OpenSpec's Living Specifications Ever Revised?

Tran Vo Thanh Nghiep Quan
Dep. of Computer and Software Engineering, Polytechnique Montréal, Montréal, Canada

***Abstract*—In spec-driven development (SDD), developers describe what to build in a Markdown specification and an AI coding agent implements it. OpenSpec, the SDD tool with the most spec files in SpecMine (the MSR 2027 Mining Challenge dataset), keeps one *living* spec per capability and calls it the source of truth. We ask whether living specs are actually maintained. In a preliminary analysis of 72,971 living specs, 62.7% were never revised after creation, and 19.6% (14,294 specs) are *fossils*: they still hold a TBD placeholder in their Purpose section, usually the one OpenSpec's archive step writes, and nobody ever edited them. We pose four research questions: how many living specs get revised or become fossils, what characterizes each group, and how the two differ.**

I. Problem Statement

SDD tools such as OpenSpec [1] and GitHub Spec Kit [2] make the specification the input of AI coding agents [3]. A *spec* is a Markdown file, written before or alongside the code, that describes what a system should do. In OpenSpec, a developer proposes a change (`changes/<name>/` with a proposal, delta specs, a design and tasks), the agent implements it, and *archiving* moves the change to `changes/archive/` and merges its delta specs into `openspec/specs/` (Fig. 1). The specs there are *living specs*: each describes how one capability of the system currently behaves and is meant to be updated whenever an archived change affects that capability. OpenSpec calls them the source of truth [1], and agents read them before every later change. A spec that stops changing while the code moves on is wrong input for the next agent.

OpenSpec gives us an unusual marker of unfinished documentation. Before v1.7.0, archiving a change that introduced a new capability always wrote "`TBD - created by archiving change <name>. Update Purpose after archive.`" into the new spec's Purpose section [4]. The fix (PR #1431) shipped on July 29, 2026, after SpecMine's crawl ended (July 11), so any spec in the snapshot could have received the placeholder. A spec that still holds it and was never revised was written once and never finished (Fig. 2); it is also stale only if its code moved on (RQ3).

Outdated documentation is a long-standing problem [5], [6], [7]. The SpecMine authors list spec lifecycle, including "placeholders never filled," as an open question [8]. Their practitioners report *spec decay*, which commit counts alone cannot show [8]; we add a tool-written marker.

Link with the course. Living specs are configuration for AI agents, kept as code; their staleness mirrors configuration drift. OpenSpec has since added a validation check for the placeholder [9], a gate a CI pipeline can enforce; we measure how often such a gate would fire on existing specs.

II. Research Questions

All analyses start from *commits per spec file*. A *revision* is a commit that touches the file after its creation commit. We separate *archive merges*, the revisions written by OpenSpec's archive step (commits that also create a file under `changes/archive/`), from other edits. An archive merge follows automatically from archiving a change, so it says little about whether anyone chose to maintain the spec; other edits are deliberate. A *fossil* is a living spec whose snapshot content holds an unfilled placeholder and that has zero revisions.

RQ1: What portion of living specs get revised, and how many become fossils? *Motivation:* if living specs are the source of truth, each archived change that touches a capability should revise its spec; the naive "never revised" rate mixes finished specs with forgotten ones, and the placeholder helps tell them apart. *Approach:* revisions per living spec, split into archive merges and other edits; the shares of fossils, of specs whose placeholder survived a revision and of specs without one, among all living specs and among specs born with the placeholder (to be decided); Wilson 95% CIs (or a bootstrap over repositories: to be decided). *Baselines:* archived specs, frozen by design (96.7% never revised), and delta specs; the naive measure; a no-placeholder control group. *Robustness:* file age: 22% of living specs are under 30 days old, so we re-run every result without specs younger than 30, 60 and 90 days (for specs at least 90 days old, 58.0% were never revised and 16.8% are fossils); repository dominance: per-repository caps and leave-largest-out.

RQ2: What characterizes living specs that are revised? *Motivation:* revised specs show what maintenance looks like in practice. *Approach:* the time to first revision (Kaplan–Meier, censored at the snapshot), who revised (human, agent or bot), whether each revision was an archive merge or another edit and, on the sample, the spec category (both defined below). For *fixed* specs (first version with the placeholder, snapshot without; found with `git log -p` on cloned repositories), we record who filled the placeholder. *Baseline:* counting archive merges as maintenance.

RQ3: What characterizes fossils? *Motivation:* a fossil may be a finished spec whose Purpose nobody filled or a forgotten one whose code moved on. *Approach:* repository activity after creation, bulk creation (other specs created in the same commit) and, on the sample, the spec category, completeness and whether the code the spec describes changed

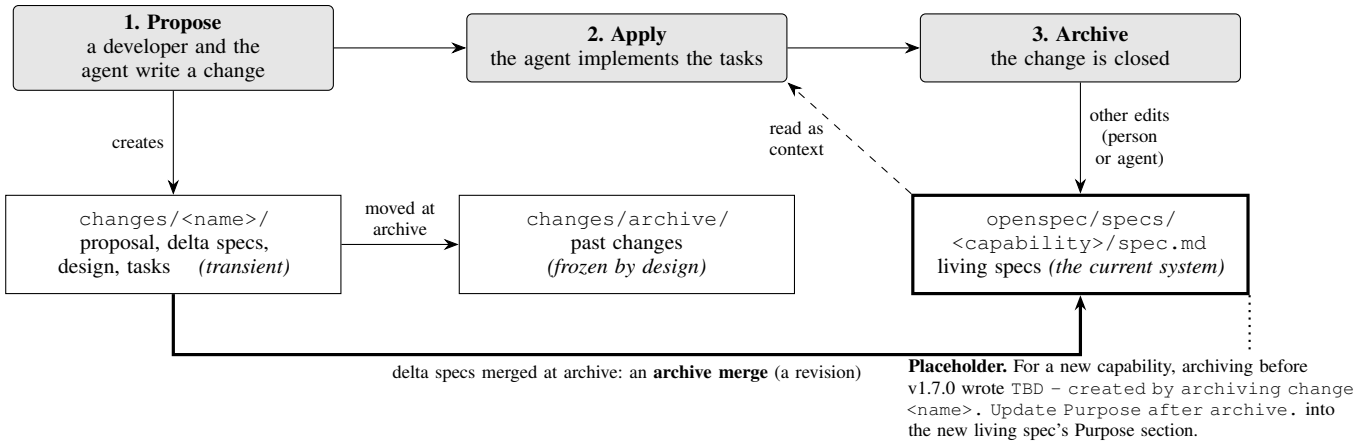


Figure 1. The OpenSpec workflow and the three folders. A revision of a living spec is either an archive merge or another edit; a fossil is a living spec that still holds the placeholder and has no revision.

```
<!-- openspec/specs/hardware-bridge-removal/spec.md -->
# hardware-bridge-removal Specification

## Purpose
TBD - created by archiving change <change>. Update Purpose
     after archive.
## Requirements
### Requirement: Hardware bridge stub is removed from the
     workspace
The repository SHALL remove the placeholder `<package>`
     package because hardware wallet support is provided
     directly by `<package>`.

#### Scenario: Stub package deleted
- **WHEN** the cleanup change is applied
- **THEN** `<path>` SHALL no longer exist in the repository

#### Scenario: Workspace no longer references stub package
- **WHEN** workspace manifests are evaluated after the
     cleanup
- **THEN** no package-manager or task-runner config SHALL
     reference `<package>`
```

Figure 2. A fossil from SpecMine: the Purpose still holds the archive placeholder, while the requirements are complete. The spec was chosen at random with a fixed seed by `src/s05_example_fossil.py`. Lightly edited: the repository, the change name, paths and package names are masked.

later. *Baselines:* never-revised specs without a placeholder; inactive repositories separate forgotten from finished specs.

RQ4: How do revised living specs and fossils differ? *Motivation:* the contrast shows which traits go with maintenance. *Approach:* we compare the two groups on the features of RQ2–RQ3 with χ^2 tests and Cramér's V, and report never-revised specs without a placeholder as a third group (to be decided). *Baseline:* the naive split into never-revised and revised specs. **Sampling and coding**. Per repository, we sample at most one fossil and one revised spec, with a fixed seed: about 375 fossils and 379 revised specs (95% confidence, $\pm 5\%$). *Spec category:* we reuse the taxonomy and judge prompts of SpecMine [8]: requirements content (problem, behaviour, acceptance, constraints, stakeholders) and implementation content (architecture, plan, code, conventions), each with an extent from none to dominant, and the kind of work (purpose: feat, fix, refactor, style, perf; object: docs, test, ci, build, chore). We add our own dimension, completeness (complete except Purpose, skeletal, near-empty). Both students code a pilot of 30 specs to check whether the taxonomy fits living specs, which describe a system as it stands, or needs extending, then double-code 100 (Cohen's κ). An LLM (temperature 0, tuned on the pilot only) labels the rest if it reaches $\kappa \geq 0.6$ on held-out human labels. A planned extension compares tools without archived specs, as only OpenSpec encodes lifecycle roles [8].

Table I
PRELIMINARY RESULTS FOR OPENSPEC LIVING SPECS (WILSON CIS)

Quantity	Value	95% CI
Living / delta / archived specs	72,971 / 41,582 / 156,372	
Living specs never revised	62.7%	[62.4, 63.1]%
Unfilled placeholder	29.2%	[28.9, 29.6]%
and never revised (fossil)	19.6%	[19.3, 19.9]%
and revised at least once	9.6%	
Repos with fossils (largest share: 0.94%)	2,488	

III. DATASET

We use SpecMine v1.0 (July 2026 snapshot) [10], [8]: 470,795 spec files from 73,030 GitHub repositories. OpenSpec accounts for 274,955 files (58.4%) in 8,926 repositories. We query the Parquet tables with DuckDB: `spec_files` (tool, lifecycle role), `spec_file_commits` (per-file history) and `spec_content_features` (the placeholder flag), plus snapshot texts (`specs.jsonl.gz`, Zenodo only). Our cohort keeps the 468,163 specs with both a fetched commit history and fetched content (735,667 commits); the 2,632 excluded files lack history (1,473) or content (2,488); 1,329 of them lack both. Table I gives our preliminary numbers. Fig. 4 shows that fossils cluster by repository: 70% of repositories with at least 3 living specs have a fossil rate at or below 10% or at or above 90%, so we sample fossils per repository.

Data caveats we already checked. (1) Histories do not follow renames: in 29 of 30 sampled specs, SpecMine's commit count equals a path-only `git log`, and 1 of the 15 living specs

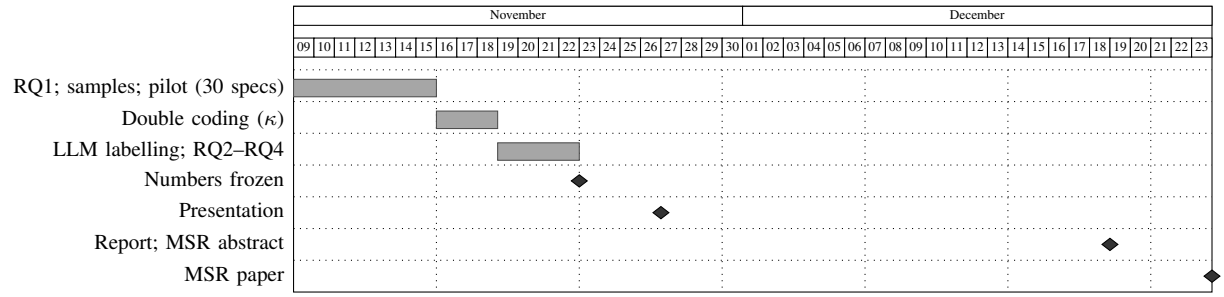


Figure 3. Plan from November 9, 2026 (dotted lines separate weeks, Monday to Sunday).

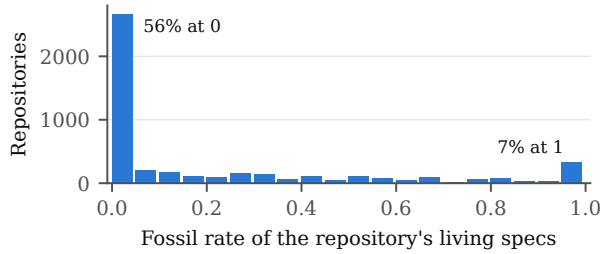


Figure 4. Fossil rate per repository ($\geq$3 living specs; 4,618 repositories).

was renamed, so revision counts are lower bounds. (2) The placeholder flag (snapshot only) catches all 19,651 specs with the archive text; 92.2% of flagged specs hold that text, 98.7% some TBD Purpose. (3) SpecMine has no code commits outside its pull-request layer, so code activity and fixed specs require cloning repositories.

IV. Related Work

SDD and SpecMine. Agarwal et al. [8] release SpecMine and study SDD through interviews and 266 repositories; they report that living specs decay but do not measure it. We do, with per-file history over all 72,971 living specs.

Outdated documentation. Documentation is often out of date [5], up-to-dateness is part of documentation-issue taxonomies [6], and outdated code references can be detected in READMEs and wikis [7]; we study specs the tool labels current.

Self-admitted technical debt. Potdar and Shihab [11] and Bavota and Russo [12] study debt that developers admit in code comments and whether it is removed. In our setting the tool writes the debt marker, and we ask whether anyone pays it.

V. Time Planning of Project

Fig. 3 shows the plan from November 9, 2026. Numbers are frozen on November 22; the course report and the MSR 2027 Mining Challenge abstract are due on December 18, and the challenge paper (4 pages plus references, double-blind) on December 23.

References

[1] Fission-AI, "OpenSpec: Repository README and Concepts documentation," 2026, accessed: Oct. 18, 2026. [Online]. Available: https://github.com/Fission-AI/OpenSpec

[2] GitHub, "Spec Kit," 2025, accessed: Oct. 18, 2026. [Online]. Available: https://github.com/github/spec-kit

[3] B. Böckeler, "Understanding spec-driven-development: Kiro, spec-kit, and Tessl," martinfowler.com, Oct. 2025, accessed: Oct. 18, 2026. [Online]. Available: https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

[4] Fission-AI/OpenSpec, "fix(archive): keep the delta spec's Purpose in a new main spec," GitHub pull request #1431, Jul. 2026, released in v1.7.0. [Online]. Available: https://github.com/Fission-AI/OpenSpec/pull/1431

[5] T. C. Lethbridge, J. Singer, and A. Forward, "How software engineers use documentation: The state of the practice," *IEEE Software*, vol. 20, no. 6, pp. 35–39, 2003.

[6] E. Aghajani, C. Nagy, O. L. Vega-Márquez, M. Linares-Vásquez, L. Moreno, G. Bavota, and M. Lanza, "Software documentation issues unveiled," in *Proc. IEEE/ACM 41st Int. Conf. on Software Engineering (ICSE)*, 2019, pp. 1199–1210.

[7] W. S. Tan, M. Wagner, and C. Treude, "Detecting outdated code element references in software repository documentation," *Empirical Software Engineering*, vol. 29, no. 1, Jan. 2024, art. no. 5.

[8] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux, "Spec-driven development as prescribed and as practiced," arXiv:2608.25202v4 [cs.SE], Oct. 2026.

[9] Fission-AI/OpenSpec, "fix(validate): report a Purpose left as the archive placeholder," GitHub pull request #1671, Aug. 2026, released in v1.12.0. [Online]. Available: https://github.com/Fission-AI/OpenSpec/pull/1671

[10] S. Agarwal and B. Vasilescu, "SpecMine: A large-scale corpus of spec-driven development artifacts (v1.0)," Zenodo, doi:10.5281/zenodo.22102780, Aug. 2026.

[11] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2014, pp. 91–100.

[12] G. Bavota and B. Russo, "A large-scale empirical study on self-admitted technical debt," in *Proc. 13th Int. Conf. on Mining Software Repositories (MSR)*, 2016, pp. 315–326.